DEFCON GROUP CASABLANCA

VMSurgeon-Gdb

Mouad Abouhali

Tech Lead @ Offsec[CDR] Deloitte FR/MA

mabouhali@deloitte.fr

(@_m00dy_)

Agenda

>_ Introduction & Context

>_ Challenges of virtual pentesting and the limits of offline disk manipulation.

>_ The Paradigm Shift

>_Virtual DMA attacks via Hypervisor GDB Stubs.

>_ The Methodology (VM-Surgeon-GDB)

>_ Reversing memory to bridge the Semantic Gap

>_ Kernel Reconnaissance & Heuristics

>_ KPTI & Context Switching (Software MMU)

>_ Solving the Paged-Out Memory Problem

>_ The Methodology (VM-Surgeon-GDB)

>_ Surgical patching of NTLM Authentication

>_Mitigations & Conclusion

>_Future Roadmap and next challenges

The Authentication Wall: Encountering an Unknown Windows VM

The Challenge of virtual pentesting

</> The Scenario

- You need to audit or pivot from a Windows Virtual Machine, but you lack valid credentials.

</> The traditional approach

- Offline disk manipulation (assuming an unencrypted VMDK/VHD).
  - Modifying the SAM database.
  - Classic "Sticky Keys" attack (e.g., replacing `sethc.exe` with `cmd.exe`).

</> Why Offline Attacks Increasingly Fail?

- Modern Windows hardens its perimeter even against physical (or offline virtual) access.
  - Secure Boot & UMCI (User-Mode Code Integrity) / WDAC: Prevent tampered or substituted binaries from executing on the lock screen.

The Trouble with Secret Extraction

</> The Data Extraction Approach

- Tools like VMKatz parse the virtual disk offline to extract secrets from hibernation files or the VM state (e.g., LSASS memory).

</> Operational Blocking Points

- **Network Isolation:**
  - Extracted hashes/tickets are useless if your attacking machine cannot route traffic to the Domain Controller (e.g., to replay Kerberos).
- **EDR Detection:**
  - Attempting Pass-the-Hash or Pass-the-Ticket inside the VM triggers massive telemetry. Modern EDRs aggressively detect and block these legacy post-exploitation techniques.
- **Result:**
  - You have the secrets, but no safe way to use them interactively.

VMkatz

license MIT | Release passing | CI passing | platform linux | windows | macos | esxi

Too Big to Steal

You are three weeks into a red team engagement. Your traffic crawls through a VPN, then bounces across four SOCKS proxies chained through compromised jump boxes before it touches the target network. Every packet takes the scenic route.

After days of lateral movement you land on a NAS attached to the virtualization cluster and the directory listing hits different: rows upon rows of `.vmdk`, `.vmsn`, `.sav`. Hundreds of gigabytes of virtual machines - domain controllers, admin workstations, the crown jewels - sitting right there.

But your link wheezes at 200 KB/s. Pulling a single 100 GB disk image would take **six days**, and every hour of sustained exfil is another chance the SOC spots the anomaly, burns your tunnel, and the whole chain collapses.

Without VMkatz, the traditional workflow looks like this: exfiltrate the entire VM disk or memory snapshot, mount it locally, install a full Windows analysis stack, load the snapshot into a debugger or use mimikatz on a booted copy, and manually piece together credentials from each VM - one at a time. Multiply that by a dozen VMs on the cluster and you are looking at days of bandwidth, tooling, and post-processing.

VMkatz exists because you shouldn't have to exfiltrate what you can read in place. It extracts Windows secrets - NTLM

Thinking Outside the Disk: Virtualized DMA Attacks

</> The Core Idea

- Bring physical attack vectors into the virtual realm.

</> Hardware DMA (Direct Memory Access)

- Traditionally performed via physical PCI or Firewire ports to hijack OS physical memory dynamically bypassing OS controls.

</> The Virtual Equivalent

- Bypass disk defences by targeting the live VM memory directly.
  - **Objective:** Dynamically locate and patch the specific LSASS/NTLM DLLs responsible for password verification directly in physical RAM.

DEMO

Following my previous research

</> Inspiration from Emuroot

- My Previous research successfully demonstrated gaining root on Android Emulators by abusing the hypervisor's GDB stub

</> Applying the Paradigm to Windows

- Hypervisors like VMware Workstation contain hidden debugging interfaces (GDB stubs).
  - Remember Windbg and Kernel debugging requires admin access to enable it .

</> VM-Surgeon-GDB

- A tool built to leverage the VMware GDB stub, granting absolute, EDR-invisible control over the guest operating system's memory.

- **Android Emuroot: Outils de rooting d'un émulateur Android Google API PlayStore** — Anaïs Gantet, Mouad Abouhali

Date : 04 juin 2020 à 09:30 — 15 min.

Un environnement de travail idéal pour l'analyse dynamique d'applications mobiles Android doit pouvoir fournir un contrôle total du système Android afin de pouvoir utiliser l'outillage nécessaire à l'analyse (déboguage, instrumentation, accès au système de fichier, etc.) ainsi que les conditions pré-requises par les applications pour fonctionner correctement (test sur plusieurs versions d'Android, besoin de chiffrement, des APIs google, etc.).

L'utilisation de l'émulateur au lieu d'un téléphone physique est le choix le plus simple pour paramétrer son environnement de travail. Actuellement Google propose plusieurs versions d'émulateur Android : des versions "eng" et "userdebug" offrant un accès root par défaut, et une version "user", aussi appelée Google API Playstore, disposant des APIs et outils Google (gmail, play store, etc.), mais dont l'accès shell est restreint. Dans le cas où l'application à analyser fonctionne uniquement sur un émulateur Google API Playstore, son analyse est rendue plus limitée puisqu'il n'est pas prévu que ce type d'émulateur soit rooté. Mais Google ne fournit pas d'émulateur fournissant à la fois un accès rooté et les API google.

Dans cette présentation, nous verrons comment Android_Emuroot répond à cette problématique, avec quelques éléments de son fonctionnement interne basés sur ADB et GDB ainsi que l'intérêt qu'il présente pour analyser des applications protégées par des mécanismes d'anti-rooting.

- Lien permanent
- Slides
- Vidéo de la présentation
- Vidéo de la présentation (basse qualité)

Android_Emuroot

Introduction

Android_Emuroot is a Python script that allows **granting root privileges** on the fly to shells running on Android virtual machines that use google-provided emulator images called *Google API Playstore*, to help reverse engineers to go deeper into their investigations.

Android_Emuroot requires the *Google API Playstore* emulator to be launched with the qemu gdb stub option enabled, and uses it to alter process metadata stored in memory in kernel task structures. The main idea is to start from a shell with the lowest privileges, then to find its associated credential structure in kernel memory to replace it by another structure having the highest privileges.

Implementing this memory modification on a *Google API Playstore* emulator presents the following two advantages:

- to work with an environment very similar to a physical device (with production build number, etc.) and with all Google Play and Google services packages already installed. This is often required by the targeted applications
- to have a very easily modifiable memory since a GDB debugger can be attached to the device to fully control the kernel memory

Prerequisites: Enabling the Debugger

</> Modifying the `.vmx` configuration

- VMware Workstation natively includes a GDB stub, but it is disabled by default. We must instruct the hypervisor to open a debugging port.

</> Add these lines to your `.vmx` file to enable debugging

```
debugStub.listen.guest64 = "TRUE"
debugStub.listen.guest64.remote = "TRUE" # Optional: allows remote IP connections
```

</> Connection

- The GDB client can now connect directly to the raw CPU state.

```
(gdb) target remote localhost:8864
Remote debugging using localhost:8864
(gdb) continue   Continuing.
^C  Thread 1 received signal SIGINT, Interrupt.
0xfffff8008388dc85 in ?? ()  (gdb) info target   Remote target using gdb-specific protocol:
(gdb) info threads     Id   Target Id              Frame
Thread 1 ( Name: VCPU 0) 0xfffff8008388dc85 in ?? ()
Thread 2 ( Name: VCPU 1) 0xfffff8008388dc85 in ?? ()
```

The Attack Chain: A Methodological Overview

</> From raw silicon to high-level domain authentication compromise.

1. Kernel Reconnaissance:

   Locating `ntoskrnl.exe` blindly.

2. Process Walking:

   Navigating undocumented structures to find `lsass.exe`.

3. Context Switching:

   Rewriting CPU registers (CR3) to enter the target process.

4. Symbolic Resolution (PDB):

   Resolving the target function dynamically.

5. Memory Patching:

   Modifying the NTLM verification logic directly in Physical RAM.

Stage 1: Finding the Windows Kernel

```
(gdb) monitor rdmsr 0xc0000082
This command is unimplemented. Please try "monitor help".
(gdb) info registers lstar
Invalid register `lstar'
(gdb)
```

</> The Problem: The GDB Stub Limitations

- When GDB connects, it only sees raw CPU registers and a flat memory model. It does not know what Windows is. It doesn't know where the OS is loaded. This is the Semantic Gap.

</> The Solution: The IDTR & `$rip` Heuristic Fallbacks

1. The MSR Failure: Ideally, we would read the MSR `IA32_LSTAR` to find `KiSystemCall64`. However, the VMware GDB stub often refuses to read MSRs (`unimplemented`)
2. The IDTR Way : Fortunately, the VMware stub natively exposes the IDT register (`monitor r idtr`). By resolving the interrupt 0x3 (`KiBreakpointTrap`), we get an address guaranteed to be inside `ntoskrnl.exe`!
3. The RIP Fallback: We interrupt the VM when the CPU is executing in Ring 0 (Kernel Mode). We take the current Instruction Pointer (`$rip`), guaranteeing we are somewhere in kernel space.
4. Scan Backwards: We Page-Align `$rip` and step back 4KB at a time until we find the `MZ` header.
5. Validate by parsing exports to confirm `NtBuildNumber` is present.

Stage 2: Listing Processes (Finding LSASS)

</> The Problem: No Process API

- We cannot run `tasklist`. We must parse internal kernel structures directly in memory.

</> The Solution: Navigating the EPROCESS List

The Solution: Navigating the EPROCESS List

1. Locate `PsInitialSystemProcess` from the kernel exports.
2. Walk the `ActiveProcessLinks` (doubly-linked list).
3. Extract the `ImageFileName` (15 bytes) until we find `lsass.exe`.

The Difficulty: Offsets for `ActiveProcessLinks` and `UniqueProcessId` change across Windows builds.

VM-Surgeon-GDB uses heuristic scanning to dynamically discover these offsets without prior knowledge.

Stage 3: The Context Switch & Virtual Memory

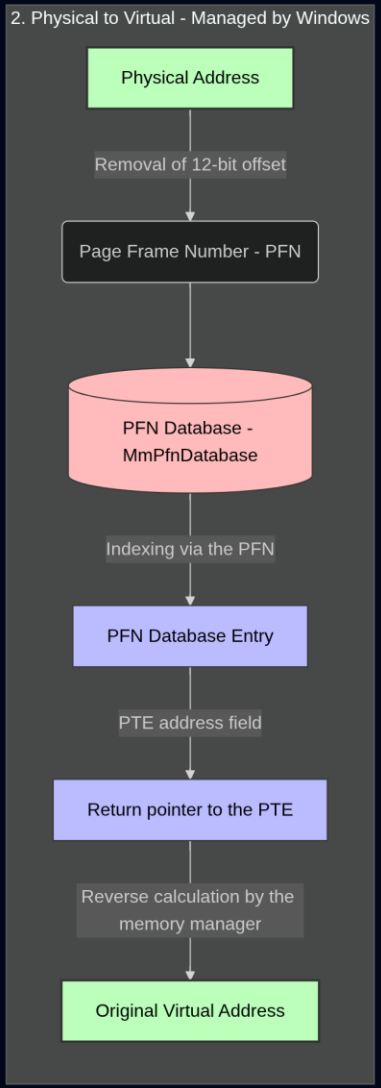
</> Theory: Virtual to Physical Address Translation

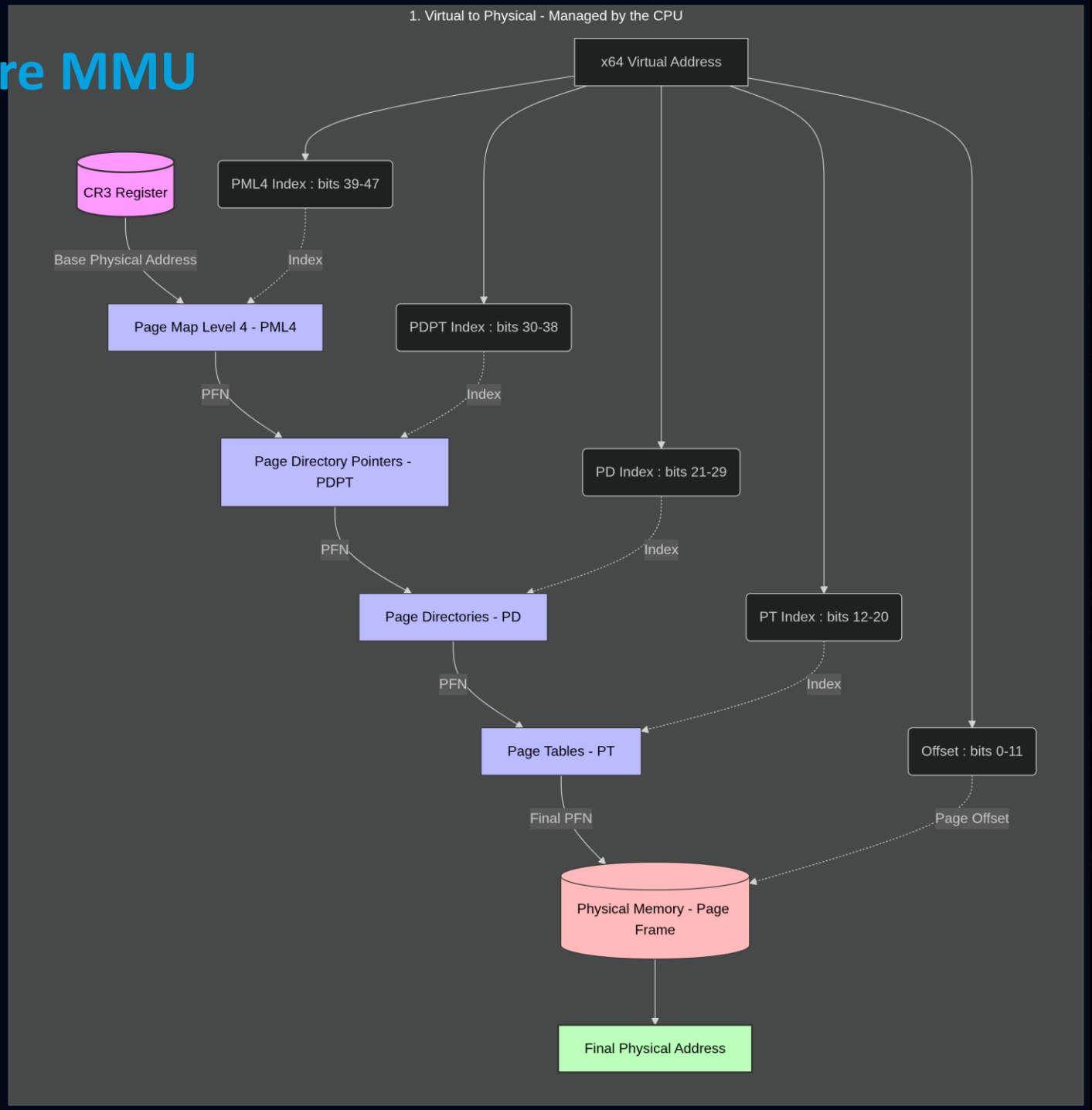
- The CPU uses the CR3 register (Directory Table Base to convert Virtual Addresses into Physical RAM using Page Tables. If GDB tries to read `lsass` memory while CR3 points to the Kernel, it reads garbage.

</> The Problem: Kernel Page Table Isolation (KPTI)

- Since Meltdown/Spectre, Windows uses KPTI.
  - There are now two CR3 registers per process: one for Kernel-Mode, one for User-Mode.
  - Solution: We scan the `EPROCESS` block heuristically to find the secondary User CR3 (often at `+0x280` or `+0x30`). Because the VMware GDB stub cannot easily overwrite CR3, we bypass it completely: the tool implements a Software MMU in Python that manually walks the hardware Page Tables to translate Virtual Addresses to Physical Addresses.
  - The GDB Stub of vmware can be switched to either VA or PA addressing (`monitor phys | monitor virt `)

Architecture: The Software MMU





Stage 4: Resolving PDBs and The Paging Problem

</> The Goal: Locating `MsvpPasswordValidate`

- We need to find `NtlmShared.dll` loaded inside LSASS and download the exact Microsoft PDB symbols to find the function's offset.

</> The Problem: Paged-Out Memory (Page Faults)

- The GDB stub does not handle page faults.
  - If the `MZ` header of `NtlmShared.dll` is swapped to the disk (pagefile), our tool crashes because the physical RAM contains zeros.
  - Solution: A hybrid approach. If we detect the memory is paged out, we read the PDB signature from a local, offline copy of `NtlmShared.dll` to query Microsoft's Symbol Server, completely bypassing the memory absence.

Stage 5: The Surgical Patch

</> The Execution

- Once the Virtual Address (VA) of `MsvpPasswordValidate` is mapped, we translate it to a Physical Address (PA) via software page-table walking.
  - We bypass caching issues by writing directly to Physical RAM.
  - The Patch:

```assembly
mov al, 1  ; Returns TRUE (Success)
ret ; Exit function ```
```

Opcodes: `\xB0\x01\xC3`

</> Result

Any NTLM authentication attempt triggers the patched function, bypassing the password check entirely.

DEMO

Future Roadmap & Next Challenges

</> Expanding VM-Surgeon-GDB's Capabilities

- Offensive Enhancements:
  - Process Manipulation: Privilege escalation, Token Integrity forging, and stripping `RunAsPPL` dynamically.
  - Payload Execution: Stealthy Kernel Driver injection and extracting LSASS secrets (hash dumping) directly from physical memory.
  - Hypervisor Portability: Extending the attack vectors to other hypervisors (Citrix XenServer, VMware vSphere/ESXi, Hyper-V, Proxmox).

</> Security Research & Analysis Tooling

- EDR/DLP Blindfolding:
  - Using the hypervisor to safely analyze security products and unhook their telemetry without detection.
  - Advanced Kernel Debugging: Implementing Time Travel Debugging (TTD), Kernel Code Coverage tracking, and Snapshot-Based Fuzzing.
  - Bootchain Subversion: Analyzing the boot sequence (with and without Secure Boot) to inject early pre-OS malicious code (Bootkits).

Questions ?